

**PROBLEMES I SOLUCIONS TROBATS DURANT LA REALITZACIÓ DEL
PROJECTE ABP.**

Índex

Descripció del Document.....	3
1. PROBLEMES I SOLUCIONS AL MÒDUL 0485. PROGRAMACIÓ.....	5
1. La raqueta no es mou.....	5
2. Col·lisions entre la pilota i la raqueta.....	6
3. Gestió del temps i puntuació.....	7
4. Dificultat Progressiva i Nivells.....	9
5. Obstacles Mòbils i Col·lisions.....	9
6. Control Dual: Mouse i teclat.....	11
7. Interfície Multiidioma (Català/Castellà/Anglès).....	12
8. Personalització Visual i Sonora.....	12
9. Bola amb disseny de Pac-Man.....	12
10. Control del creixement de la dificultat.....	13
2. PROBLEMES I SOLUCIONS AL MÒDUL 0486. ACCÉS A DADES.....	15
1. Gestió de l'estat de la connexió.....	15
2. Configuració de la IP i xarxa.....	15
3. Encapsulació de la classe Connexió.....	16

Descripció del Document

Aquest document té com a finalitat documentar els problemes i solucions que hem trobat durant la realització del projecte ABP. El document presenta pantallazos i solucions plantejades que hem proposat com a grup.

 INS Gabriela Mistral	data d'aprovació:	APROVAT	Pàgina 3 de 15
	29/04/2026	05	

1. PROBLEMES I SOLUCIONS AL MÒDUL 0485. PROGRAMACIÓ

1. La raqueta no es mou

En el context del tutorial, s'afegeix un `KeyListener` per capturar les tecles.

El Problema que ens hem trobat ha sigut que en executar el programa, les tecles no funcionen fins que es fa clic amb el ratolí dins de la finestra.

La Solució que hem proposat ha sigut afegir el mètode **`setFocusable(true)`** al constructor de la classe principal. Això indica a Java que el nostre tauler de joc ha de tenir la prioritat per rebre les entrades del teclat des del moment en què s'obre.

CODI TUTORIAL	CODI AMB SOLUCIÓ I MILLORA
<pre> @SuppressWarnings("serial") public class KeyboardExample2 extends JPanel { public KeyboardExample2() { KeyListener listener = new KeyListener() { @Override public void keyTyped(KeyEvent e) { } @Override public void keyPressed(KeyEvent e) { System.out.println("KeyPressed-"+KeyEvent.getKeyText(e.getKeyCode())); } @Override public void keyReleased(KeyEvent e) { System.out.println("KeyReleased-"+KeyEvent.getKeyText(e.getKeyCode())); } }; addKeyListener(listener); setFocusable(true); } </pre>	<pre> 66 // Classes Anònimes Internes per a la gestió d'esdeveniments de perifèrics 67 addKeyListener(new KeyAdapter() { 68 @Override 69 public void keyPressed(KeyEvent e) { 70 racquet.keyPressed(e); // Delegació de l'esdeveniment a l'objecte racquet 71 } 72 73 @Override 74 public void keyReleased(KeyEvent e) { 75 racquet.keyReleased(e); 76 } 77 }); 78 // Adaptador per a la gestió de moviment del ratolí 79 addMouseListener(new MouseMotionAdapter() { 80 @Override 81 public void mouseMoved(MouseEvent e) { 82 racquet.setMouse(e.getX()); 83 } 84 }); 85 // Habilita el focus del component per rebre esdeveniments de teclat 86 setFocusable(true); 87 // Invocació de mètode per iniciar el flux d'àudio de fons 88 sonido.playFondo(); 89 90 } </pre>

`setFocusable(boolean)` a Java (Swing/AWT) és un mètode de classe `Component` que determina si un component gràfic (com `JPanel`, `JButton`) pot rebre el focus d'entrada del teclat. En establir-lo en `true` (per defecte en components interactius) o `false`, es controla si l'usuari pot navegar-hi usant `Tab` o clics.

2. Col·lisions entre la pilota i la raqueta

El tutorial detecta les col·lisions mitjançant rectangles. Quan la pilota toca la raqueta, a vegades el programa detecta el xoc diverses vegades seguides abans que la pilota pugui sortir de l'àrea, fent que reboti de manera errònia cap a dins de la raqueta.

Per tant, s'ha implementat una correcció de posició forçada. En el moment de la col·lisió, a part d'invertir la velocitat (***-Math.abs(yVel)***), s'assigna a la pilota una coordenada **Y** exactament per sobre de la raqueta:

nextY = game.racquet.getTopY() - DIAMETER;

CODI TUTORIAL

```
@SuppressWarnings("serial")
public class Game extends JPanel {

    int x = 0;
    int y = 0;
    int xa = 1;
    int ya = 1;

    private void moveBall() {
        if (x + xa < 0)
            xa = 1;
        if (x + xa > getWidth() - 30)
            xa = -1;
        if (y + ya < 0)
            ya = 1;
        if (y + ya > getHeight() - 30)
            ya = -1;

        x = x + xa;
        y = y + ya;
    }
}
```

Math.abs() és un mètode integrat a Java que retorna el valor absolut d'un argument determinat. Converteix nombres negatius en positius, facilitant càlculs i comparacions.

3. Gestió del temps i puntuació

Inicialment, el tutorial d'*edu4java* proposava una puntuació per cops o punts fixos. L'enunciat demanava que la puntuació fos el **temps transcorregut en mil·lisegons**.

S'ha implementat una variable ***startTime*** que captura el ***System.currentTimeMillis()*** al començar. La puntuació es calcula dinàmicament restant el temps inicial del temps actual en cada frame. Es mostra el temps en mil·lisegons directament a la pantalla (***g2d.drawString()***) per donar feedback constant a l'usuari.

CODI TUTORIAL

```
private int getScore() {  
    return speed - 1;  
}
```

Para pintar la puntuación en el rincón superior izquierdo al final del método paint he agregado:

```
g2d.setColor(Color.GRAY);  
g2d.setFont(new Font("Verdana", Font.BOLD, 30));  
g2d.drawString(String.valueOf(getScore()), 10, 30);
```

En la primera línea elegimos el color gris, en la segunda línea el tipo de letra Verdana, negrita de 30 píxeles y finalmente en la posición (x,y) igual a (10,30) donde dibujamos la puntuación.

En el método gameOver() modificamos el segundo parámetro para mostrar la puntuación alcanzada:

```
JOptionPane.showMessageDialog(this, "your score is: " + getScore(),  
    "Game Over", JOptionPane.YES_NO_OPTION);
```

CODI AMB SOLUCIÓ I MILLORA

```
36     private long score = 0;  
37     private long startTime = System.currentTimeMillis();  
38     private long lastPointUpdate = System.currentTimeMillis();
```

```

96 private void updateLevel() {
97     long now = System.currentTimeMillis();
98
99     // Lògica d'ACUMULACIÓ DE PUNTS
100    // Calculem quants mil·lisegons han passat des de l'últim "update"
101    long elapsed = now - lastPointUpdate;
102    score += elapsed; // Acumulem el temps a la puntuació total (no es reinicia mai)
103    lastPointUpdate = now; // Actualitzem la marca de temps per al pròxim càlcul
104
105    // Lògica de canvi de nivell
106    // Controla si han passat 20 segons des de l'últim canvi de nivell
107    if (now - startTime >= 20000) {
108        level++;
109        startTime = now;
110        ball.increaseSpeed();
111        actualitzarObstacles(level);
112    }
113 }
114
115 protected void paintComponent(Graphics g) {
116     super.paintComponent(g);
117     g.drawImage(fondo, 0, 0, getWidth(), getHeight(), this);
118
119     Graphics2D g2d = (Graphics2D) g;
120     g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
121
122     // Pintem els objectes
123     ball.paint(g2d);
124     racquet.paint(g2d);
125     for (Obstacle o : obstacles) {
126         o.paint(g2d);
127     }
128
129     // renderizat de text
130     g2d.setColor(Color.WHITE);
131     g2d.setFont(new Font("Retro", Font.BOLD, 12));
132
133     // Mostrem el Nivell actual
134     g2d.drawString("Level: " + level, 10, 20);
135
136     // Mostrem els Punts Acumulats
137     g2d.drawString("Points: " + score + " ms", 10, 40);
138 }
  
```

4. Dificultat Progressiva i Nivells

El joc havia de pujar de nivell cada 20 segons, augmentant la velocitat un 10%. Es va crear un mètode `updateLevel()` dins de la classe `Game`. Aquest mètode controla si han passat 20.000 ms per incrementar la variable `level` i cridar a **`ball.increaseSpeed()`**.

CODI AMB SOLUCIÓ I MILLORA

```
30-  /**
31      * Mètode que incrementa la velocitat de la pilota en un 10%.
32      */
33-  public void increaseSpeed() {
34      speed *= 1.10;
35  }
36
105      // Lògica de canvi de nivell
106      // Controla si han passat 20 segons des de l'últim canvi de nivell
107-  if (now - startTime >= 20000) {
108      level++;
109      startTime = now;
110      ball.increaseSpeed();
111      actualitzarObstacles(level);
112  }
113  }
```

5. Obstacles Mòbils i Col·lisions

Per fer el joc més desafiant, calia introduir elements on la pilota pogués rebotar (mòbils o fixes) a mesura que augmentava el nivell. S'ha creat la classe **`Obstacle`** amb moviment horitzontal propi (`move()`).

Tot i això, la col·lisió de la pilota amb els obstacles era complexa perquè la pilota podia quedar "atrapada" dins de l'objecte. Per tant, s'ha implementat un sistema de **penetració mínima** en la classe `Ball` per detectar per quina cara (nord, sud, est, oest) xocava la pilota i així invertir la velocitat (`xVel` o `yVel`) correctament.

CODI AMB SOLUCIÓ I MILLORA

```

34-  /**
35   * Mètode que, implementa la lògica de moviment horitzontal.
36   */
37-  public void move() {
38      int width = game.getWidth();
39-      if (width > 0) {
40          x += dx;
41-          if (x <= 0 || x >= width - WIDTH) {
42              dx = -dx;
43          }
44      }
45  }

83  // Càlcul de la penetració de l'objecte per cada costat per determinar la direcció del rebot
84  int penetracioDreta = ballRect.x + ballRect.width - obsRect.x;
85  int penetracioEsquerra = obsRect.x + obsRect.width - ballRect.x;
86  int penetracioTerra = ballRect.y + ballRect.height - obsRect.y;
87  int penetracioSostre = obsRect.y + obsRect.height - ballRect.y;
88  // Busquem la penetració mínima per saber quina cara de l'obstacle s'ha tocat
89  int penetracioMinima = Math.min(Math.min(penetracioDreta, penetracioEsquerra), Math.min(penetracioSost
90  // Inversió de velocitat i ajust de posició segons el punt de xoc
91-  if (penetracioMinima == penetracioDreta) {
92      xVel = -Math.abs(xVel);
93      nextX = obsRect.x - ballRect.width;
94  }
95-  else if (penetracioMinima == penetracioEsquerra) {
96      xVel = Math.abs(xVel);
97      nextX = obsRect.x + obsRect.width;
98  }
99-  else if (penetracioMinima == penetracioTerra) {
100     yVel = -Math.abs(yVel);
101     nextY = obsRect.y - ballRect.height;
102 }
103-  else if (penetracioMinima == penetracioSostre) {
104     yVel = Math.abs(yVel);
105     nextY = obsRect.y + obsRect.height;
106 }
107
108 game.sonido.playGolpe(); // So del choque amb l'obstacle
109 break; // Sortim del bucle per gestionar només una col·lisió
110

```



data d'aprovació:

29/04/2026

APROVAT

05

Pàgina 9 de 15

6. Control Dual: Mouse i teclat

El tutorial només cobria el teclat, però per a un joc de frontó, el ratolí ofereix molta més precisió.

Per tant, hem afegit un **MouseMotionAdapter** a la classe Game. La classe Racquet ara té el mètode **setMouse(int mouseX)** que permet centrar la raqueta a la posició del cursor, mantenint el control per teclat com a alternativa.

CODI AMB SOLUCIÓ I MILLORA

```
Game.java × Ball.java InitialWindow.java Obstacle.java Racquet.java MainApp.java Sound.java Controllanguage.java
66 // Classes Anònimes Internes per a la gestió d'esdeveniments de perifèrics
67 addKeyListener(new KeyAdapter() {
68     @Override
69     public void keyPressed(KeyEvent e) {
70         racquet.keyPressed(e); // Delegació de l'esdeveniment a l'objecte racquet
71     }
72 }
73
74 @Override
75 public void keyReleased(KeyEvent e) {
76     racquet.keyReleased(e);
77 }
78
79 // Adaptador per a la gestió de moviment del ratolí
80 addMouseMotionListener(new MouseMotionAdapter() {
81     @Override
82     public void mouseMoved(MouseEvent e) {
83         racquet.setMouse(e.getX());
84     }
85 });
86 // Habilita el focus del component per rebre esdeveniments de teclat
87 setFocusable(true);
88 // Invocació de mètode per iniciar el flux d'àudio de fons
89 sonido.playFondo();
90 }
91
92 public void setMouse(int mouseX) {
93     // Càlcul per centrar l'objecte: es resta la meitat de l'amplitud a la posició del cursor
94     int newX = mouseX - (WIDTH / 2);
95     int limit = (game.getWidth() <= 0) ? 300 : game.getWidth();
96     // Validació de seguretat per mantenir l'objecte dins de l'espai renderitzable
97     if (newX > 0 && newX < limit - WIDTH) {
98         this.x = newX;
99     }
100 }
```

7. Interfície Multiidioma (Català/Castellà/Anglès)

El joc havia de ser totalment multiidioma des de l'inici. S'ha dissenyat la classe **ControlLanguage** que actua com un diccionari. I en lloc de posar el text directament al codi, es crida a **controlLang.get("clau")**, facilitant que en el futur es puguin afegir més idiomes.

8. Personalització Visual i Sonora

El joc base era molt simple visualment (fons negre i colors bàsics). S'han afegit imatges de fons (**fondovideojuego.jpg**) mitjançant **Imagelcon**. També s'ha creat la classe **Sound** per gestionar la música de fons en bucle i els efectes de so en els xocs i el *Game Over*.

9. Bola amb disseny de Pac-Man

L'aspecte visual de la pilota era inicialment un cercle groc bàsic dibuixat mitjançant codi (fillOval). Tot i que funcionava, l'estètica era massa simple i el nostre joc està inspirat en el videojoc Pac-Man. A més, en intentar substituir el cercle per una imatge externa, s'ha detectat un problema de **pixelació** (les vores es veien dentades i borroses) i la mida inicial de 30 píxels feia que el personatge no es visualitzi prou bé en pantalla.

Per tant, hem aplicat les següents solucions:

1. **Refactorització del mètode de dibuix:** S'ha substituït la funció fillOval per drawImage a la classe Ball, integrant un fitxer d'imatge .png transparent per evitar el fons blanc.
2. **Escalat i Resolució:** S'ha augmentat el diàmetre de l'entitat de 30 a 50 píxels per millorar-ne la visibilitat.
3. **Aplicació de Rendering Hints (Suavitzat):** Per eliminar la pixelació, s'ha configurat el context gràfic Graphics2D amb tècniques d'**Anti-aliasing**. Això fa que Java calculi els píxels intermedis al redimensionar la imatge, oferint un acabat net

 INS Gabriela Mistral	data d'aprovació:	APROVAT	Pàgina 11 de 15
	29/04/2026	05	

CODI AMB SOLUCIÓ I MILLORA

```

127  /**
128   * Mètode que s'encarrega de dibuixar la pilota en el context gràfic.
129   */
130  public void paint(Graphics2D g) {
131      g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
132      g.setRenderingHint(RenderingHints.KEY_INTERPOLATION, RenderingHints.VALUE_INTERPOLATION_BILINEAR);
133      g.drawImage(imagenBall, x, y, DIAMETER, DIAMETER, null);
134  }
135
136
137  private Image imagenBall; // Variable per guardar la imatge
138  private static final int DIAMETER = 50;
139
140  /**
141   * Constructor, que inicialitza la pilota connectant-la amb la instància del joc
142   * actual.
143   */
144  public Ball(Game game) {
145      this.game = game;
146      this.imagenBall = new ImageIcon(getClass().getResource("pacman-removebg-preview.png")).getImage();
147  }
  
```

10. Control del creixement de la dificultat

En implementar nivells infinits, la velocitat de la pilota pujava un 10% a cada nivell de forma exponencial. Arribava un punt on la pilota anava tan ràpida que "traspassava" la raqueta en un sol frame (efecte túnel) i el joc es tornava impossible de processar i de jugar.

Per tant, hem implementat un sistema de limitatge. Hem definit una constant **MAX_BALL_SPEED** (valor de 12.0). La velocitat es calcula amb la fórmula **Math.min(velocitatCalculada, MAX_BALL_SPEED)**, assegurant que el joc sigui reptador però tècnicament estable.

 INS Gabriela Mistral	data d'aprovació:	APROVAT	Pàgina 12 de 15
	29/04/2026	05	

CODI AMB SOLUCIÓ I MILLORA

```
61  /**
62   * Mètode que incrementa la velocitat de la pilota
63   */
64  public void increaseSpeed() {
65
66      speed *= INCREMENT_VELOCITAT;
67      // Calculem la nova velocitat potencial
68      double nuevaVelocidad = speed * Utils.INCREMENTO_POR_NIVEL;
69
70      // Solo aumentamos si no hemos llegado al límite de frames
71      if (nuevaVelocidad <= Utils.MAX_BALL_SPEED) {
72          speed = nuevaVelocidad;
73      } else {
74          speed = Utils.MAX_BALL_SPEED;
75      }
76  }
77  }
```

2. PROBLEMES I SOLUCIONS AL MÒDUL 0486. ACCÉS A DADES

1. Gestió de l'estat de la connexió

El problema ha sigut que si la base de dades perdia la connexió per un temps d'espera o si s'intentava fer una operació amb la connexió tancada per l'error anterior, el programa llançava una **SQLException** i el joc es bloquejava o no guardava les dades.

S'ha creat un mètode anomenat **connectar()** que es crida al principi de cada operació (guardarPartida o consultarRanking). Aquest mètode verifica si la connexió és nul·la o si s'ha tancat (`cn.isClosed()`) i, només en aquest cas, torna a obrir-la.

```
--  
20  /**  
21   * Mètode privat per carregar el controlador (driver) i obrir la connexió.  
22   * Gestiona la reobertura si la connexió s'ha perdut o tancat.  
23   */  
24  private void connectar() {  
25      try {  
26          // Comprova si la connexió és nul·la o s'ha tancat per seguretat  
27          if (cn == null || cn.isClosed()) {  
28              // Carrega dinàmicament el Driver de MySQL  
29              Class.forName("com.mysql.cj.jdbc.Driver");  
30              // Defineix la ruta de la BD, l'usuari i la contrasenya (contrasenya de MySQL candela)  
31              cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/retro_tennis", "root", "7022");  
32          }  
33      } catch (Exception ex) {  
34          // Captura qualsevol error (Driver no trobat o error de xarxa)  
35          System.out.println("Error al connectar: " + ex.getMessage());  
36      }  
37  }
```

2. Configuració de la IP i xarxa

Inicialment s'estava utilitzant una IP específica (la de la connexió de la màquina virtual), que causava errors de "Connection Refused" si el servidor no estava correctament encès.

Per tant, el que hem proposat ha sigut posar la base de dades a una de les màquines físiques (d'un membre del grup: candela), posar el port que tingui connectat al MySQL, el seu nom d'usuari i la seva contrasenya.

 INS Gabriela Mistral	data d'aprovació:	APROVAT	Pàgina 14 de 15
	29/04/2026	05	

3. Encapsulació de la classe Connexió

Inicialment, la nostra classe **Connexió**, on es troba la configuració, les insercions i la visualització de consultes, no seguia estrictament les bones pràctiques del paradigma de la **Programació Orientada a Objectes (POO)**, ja que mancava d'una gestió centralitzada del recurs (anteriorment un patró *Singleton*).

Per solucionar-ho, hem centralitzat la instància de **Connexió** dins de la classe **Game**. D'aquesta manera, el motor del joc en gestiona una única instància durant tot el seu cicle de vida, evitant obertures i tancaments innecessaris. Gràcies a aquesta reestructuració, la comunicació amb la base de dades és ara més robusta: es garanteix el guardat de resultats, s'eviten talls en la via de comunicació i es gestiona el rànquing de forma automàtica i eficient.

 INS Gabriela Mistral	data d'aprovació:	APROVAT	Pàgina 15 de 15
	29/04/2026	05	